



Development of Modern Data Platform using Medallion Architecture

Michal Wiselka

Haaga-Helia University of Applied Sciences
Degree Programme in Business Information Technology
Bachelor's Thesis
2024

Author(s) Michal Wiselka
Degree Bachelor of Business Administration
Report/Thesis Title Development of Modern Data Platform using Medallion Architecture
Number of pages and appendix pages 38 + 2
Worldwide data amounts continue to grow, forcing organizations to develop modern scalable data solutions. This thesis is focused on development of a modern data platform, using Medallion Architecture for company specialised in manufacturing of sports devices. The objective of this thesis is to further develop data platform, by integrating new data sources, enhancing analytical capabilities of internal data team. The author also wants to familiarize reader with data engineering concepts and showcase practical implementation of Medallion Architecture. The first chapters introduce reader to background theoretical concepts, as well as the Medallion Architecture itself. Reader is also walked through the overview of the solution implementation, where certain technologies are described, and their usage within solution is explained. The main part of the thesis describes implementation of the solution, the detailed journey of data from source systems to business use-cases is described. Every step, beginning from data ingestion, through data storage and data processing, ending at data governance is shown and described in detail. This part also includes deep dive into core technologies, that are a backbone of this solution: Apache Spark and Delta Lake. The thesis concludes, by evaluating the set objectives and reflections of the author about solution and technologies used. The author shares positive and negative aspects of the implementation, as well as possible improvements that should be made.
Key words Medallion Architecture, Data Engineering, Delta Lake, Data Warehousing, Data Platform

Table of contents

1	Introduction	3
1.1	Basic concepts	5
2	Medallion Architecture data platform, as modern data warehousing solution	6
2.1	Traditional data warehouse	6
2.2	Modern data warehouses	8
2.3	Medallion Architecture, as a modern Data warehousing solution	8
3	Medallion Architecture	9
3.1	Bronze layer	9
3.2	Silver layer	10
3.3	Gold layer	11
4	Solution overview	13
4.1	Introduction	13
4.2	High-level overview	13
4.2.1	Data ingestion	13
4.2.2	Data processing	15
4.2.3	Serving and governance	16
5	Solution implementation	17
5.1	Data ingestion layer	17
5.1.1	Mobile applications events	17
5.1.2	SQL server	19
5.2	Data processing layer	20
5.2.1	Apache Parquet and Delta Lake	20
5.2.2	Apache Spark and Databricks	24
5.2.3	Data processing on the Data platform	25
5.2.4	Mobile application events processing	26
5.2.5	SQL Database processing	30
5.3	Serving and data governance layer	32
5.3.1	Access control	32
5.3.2	Data consumption	33
6	Discussion	34
6.1	Objectives	34
6.2	Medallion Architecture evaluation	34
6.3	Reflections on implementation	35
6.4	Further studies	35

6.5 Learnings & self-evaluation	36
References	37
Appendices	39
Appendix 1. List of figures	39

1 Introduction

The total amount of data collected worldwide grows exponentially, in 2015 the total amount of data collected was 15.5 zettabytes, and at the end of the 2024 we are projected to have collected 127 zettabytes. (IDC 2024) Growing data amounts bring new opportunities for the organizations, it allows them to make better decisions and provide deeper business insights. Furthermore, nowadays many companies embark on machine learning, artificial intelligence journeys, which need to be fueled by enormous volumes of data. All of this is to say that there are plenty of use cases, the issue is how to handle all these data. The field that is responsible for handling data understood as capturing, transforming, storing is called data engineering.

There are many data engineering patterns and architectures. The focus of this thesis is modern data platform implemented using Medallion Architecture. Modern data platform can be defined as a centralized location for: data ingestion, cleansing, transformation, storage, and analysis. It's supposed to be single source of truth, allowing business users to conduct self-service analysis with full confidence in the data. In today's age, those data platforms have become inseparable part of running the business, as data driven decision making is a necessity.

This thesis shows modern data platform, implemented using Medallion Architecture in company specialized in manufacturing of sports watches, and development of mobile application connected to physical watches. The company in question, has growing analytical and integration needs, which need to be met, to support vital components of the business. Components for which data insights are essential include product development, feature usage reporting, financial reporting, manufacturing systems, strategic decision making. The company also wants to increase its market share, by accurate and well-designed marketing campaigns. Well-targeted marketing campaigns require excellent insights; such insights can only be derived from combining different data sources. In order to unite different data sources, a centralized data platform is required.

Objective of this thesis is to integrate needed data sources into, an in-house data platform, enhancing analytical capabilities of the organization. Integrating data sources, also means making them usable for deriving valuable insights, which requires processing the data. There are different data sources, that are needed from the business point of view. There are two particular ones that were chosen by the author, as a focus of this thesis.

Mobile application events, the events produced by sports devices and mobile application itself, are one of the data sources that need to be integrated. Mobile events were not integrated into data platform, as a third-party platform was previously used to handle this data. The goal is not only to integrate mobile events into data platform, but also at the same time reduce overall operation costs,

by lowering the usage of the third-party systems. The other data source is SQL database containing information regarding repairs of the devices.

As the data platform is implemented using Medallion Architecture, this thesis will explore the theoretical architecture patterns of Medallion Architecture and showcase practical implementation of it. Out of scope is detailed explanation of all the basic data engineering concepts.

1.1 Basic concepts

Data warehouse: centralized data repository designed for huge-volume storage of historical data, optimized data analytic workloads.

ETL: Extract Transform Load a process involving extraction of data from source systems, transformation of the data, and loading it into a final destination.

ELT: Extract Load Transform a process involving extraction of data from source systems, loading data into a staging location and transforming it into a final format.

Data pipeline: automated process of executing and orchestrating data transformations and data movements.

Data Lake: a cloud data repository designed for storage of large quantities of unstructured, semi-structured and structured data. It is made for scaling and handling many concurrent transactions.

Parquet: a columnar data format designed for efficient data storage and retrieval. Made for big data workloads.

Delta Lake: a storage solution built on top of Apache Parquet, provides additional efficiency improvements and quality-life features.

Azure: Microsoft's cloud platform offering cloud computing and storage services.

Databricks: a cloud computing and analytics platform. Databricks provisions compute for data processing, provides user interface for data operations.

PII: Personal Identifiable Information, an information that can lead to identification of an individual.

Azure Data Factory (ADF): is one of the services in Azure cloud platform. ADF is used to extract data from source systems and orchestrate data pipelines.

Azure subscription: a container for cloud resources, within Azure cloud platform. Each subscription is a separate entity allowing for isolation of resources.

Data retention policies: a policies and processes that ensure that data is retained and removed at the right time. Those policies are implemented to manage storage costs and comply with data privacy laws.

ACID quarantines: Atomicity, Consistency, Isolation, Durability, properties of storage solutions ensuring that transactions are isolated, processed reliably and data remains accurate.

2 Medallion Architecture data platform, as modern data warehousing solution

Data warehousing is a discipline that aims to prepare data and make it accessible for consumption, primarily for business intelligence. Point of the enterprise data warehouse is to gather all the needed data into one centralized location, making data from different operational system available for analysis and comparisons. The development of data platform described in this thesis aims to create a solution, which would yield similar results.

2.1 Traditional data warehouse

Traditional data warehouses are designed to support large-volume analytical workloads and long-term trend monitoring. Data warehouses enforce strict schema, and data needs to follow pre-defined structure. As traditional data warehouses are built on top of databases, the data is stored in table format.

The data within data warehouses is modelled in particular order, one of the widely used data modelling designs is a star schema. Star schema consists of dimensions tables and central fact table. The fact table contains facts such as quantities and amounts, the dimension tables can contain information like users, products and its properties (Figure 1). In general star schema is a popular data modelling design, as it is simple to understand and use. The queries run on such data model tend to be efficient, moreover star schema is well suited, for many common data analytics workloads. (Karmani et al. 2020, 5)

Star schema

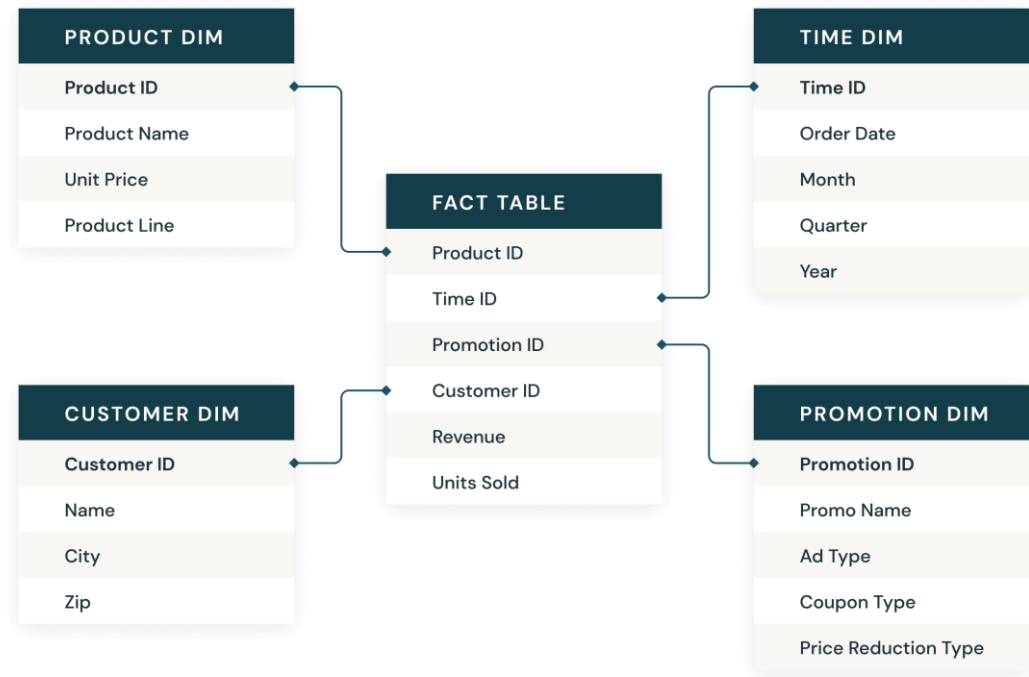


Figure 1. Example star schema. (Databricks s.a.)

Organizations have many different operational systems, and the data within them is stored in many different ways. An example can be an SAP system, which contains core business data in tables that have hundreds of columns. Each of the source systems is different and data within those systems, might be stored in different ways. The data needs to be extracted from those systems and transformed to fit the chosen data model at the data warehouse level. The process that is responsible for Extracting Transforming and Loading the data into data warehouse is called ETL. The data first needs to be extracted from source systems, then transformed, which often includes data types transformations, business calculations, duplication of data. The result of transformation process is data, that supports business use-cases.

Most traditional data warehouses are on-premises solutions, built on top of databases like Microsoft SQL server, DB2. Those databases are transactional databases, because of that among other reasons, traditional data warehouses are not as efficient as modern cloud-based equivalents.

2.2 Modern data warehouses

As mentioned above, traditional data warehouses are powerful solutions, however they do have their drawbacks and limitations. Huge planning overhead is one of them, as traditional data warehouses are complex structures requiring in-advance capacity allocation and planning. They have long deployment time, and are inflexible, the resources cannot be scaled up or down whenever needed. Furthermore, they are hard and expensive to scale. Traditional data warehouses also limit what kind of data is stored, as only structured data is supported. (Rehman, Ahmad & Mahmood, 2018, 36)

Modern data warehouses address most of the limitations of traditional ones. Modern data warehouses in most cases are cloud based. Cloud data warehouses do not require planning overhead, as the compute and storage capabilities can be easily scaled. Furthermore, cloud data warehouses are flexible and can scale up and scale down, based on the demand. Examples of modern cloud-based data warehouses include Snowflake, AWS Redshift. (Rehman, Ahmad & Mahmood, 2018, 36)

Furthermore, modern data warehouses bring efficiency improvements, one of the factors influencing efficiency improvements is column-based storage system. Unlike traditional data warehouse built on top of row-based transnational databases, modern data warehouses are built on top of column-based system. Inner workings of column-based system are described in more detail in chapter 5.2.1, what is important that column-based systems are more efficient for read intensive workloads.

2.3 Medallion Architecture, as a modern Data warehousing solution

The basic idea of a Medallion Architecture is to have accessible data, at different stages of transformation process. The first layer contains raw unchanged data, and the final one, data that has the same structure, as in the data warehouse. Meaning data has a strict schema; data is modelled according to selected pattern, and is ready to be used for business intelligence use cases.

Most of the Medallion Architecture implementations are implemented using Delta Lake storage solution. This solution is built on top of Data Lake, a large-scale data storage repository, offered by cloud providers. Those solutions are described in-detail in chapters 4.1, 5.2.1 respectively. Because Medallion Architecture implementations are cloud based, they have the same advantages over traditional data warehouses, as the cloud data warehouses. What is specific for Medallion Architecture is a layered structure for incremental data quality. Which allows easier tracking of transformations made, and simplified data governance.

3 Medallion Architecture

Medallion Architecture is a data design pattern with its core idea being to logically organize data into layers. Classic Medallion Architecture consists of three layers: bronze, silver, gold. Data is processed, cleaned, and moved between layers using data pipelines. The quality and structure of data increases from layer to layer. The bronze layer contains raw data that has been injected from the source system; silver layer has the bronze data cleaned transformed, and the gold layer has use-case specific data (Figure 2). Such an architecture pattern allows to recreate data from the raw layer at any given point. If there would be a mistake made, and some data would be removed, it can always be recreated. Furthermore, it allows easier access control, allowing certain groups of people to access only certain layers. One use-case could be handling Personally Identifiable Information (PII). This kind of data can stay in a highly restricted bronze layer, and sensitive information could be redacted, or omitted in further layers, to which business users might have access to. (Databricks 2024a, Oldham 2023)

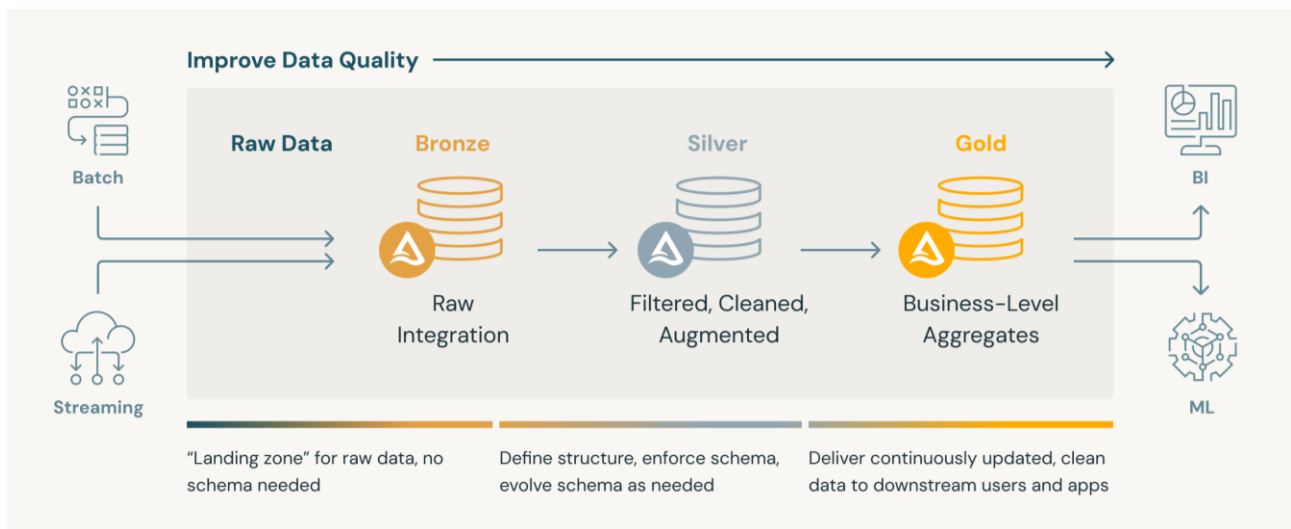


Figure 2. Medallion architecture. (Databricks 2024 b)

3.1 Bronze layer

Bronze layer contains raw unchanged data. There are no data types set, data is the same as in the source system. All the data on the remaining layers can be recreated, if the data on the bronze layer exists. Often this layer contains meta-data such as: timestamp when the data has been inserted, original file name or streaming source name.

The bronze layer can contain multiple tables within it, each of them containing data at different transformation stage. Figure 3 shows how first table within a bronze layer could look like. Every row contains an entire json file, therefore at this stage, each row contains multiple events. Meta-data about the original file, and time when it has been read from the landing location is also included in this table.

	A ^B event	🕒 timestamp	A ^B source_file_name
1	> [{"Device Id": "iot_device_001", "Device Type": "A01", "timestamp": "2024-09-03T12:00:00Z", "location": {"latitude": 37.7749, "longitude": ...	2024-09-12T13:42:19.912+00:...	file1.json

Figure 3. Bronze layer table, with contents of entire json file in each row.

Figure 4 shows a second table within a bronze layer, this table is based on the previous one. It has data that was originally in the same row, split into separate rows, each individual event is in a separate row.

	A ^B exploded_event	🕒 timestamp	A ^B source_file_name
1	> [{"Device Id": "iot_device_001", "Device Type": "A01", "timestamp": "2024-09-03T12:00:00Z", "location": {"latitude": 37.7749, "longitude": -122.4194, "temperature": 25.6, "humidity": 60.2, "pressure": 1013.25}	2024-09-12T13:43:02.472+00:...	file1.json
2	> [{"Device Id": "iot_device_002", "Device Type": "A02", "timestamp": "2024-09-03T12:05:00Z", "location": {"latitude": 34.0522, "longitude": -118.2437, "temperature": 24.1, "humidity": 65.1, "pressure": 1015.3}	2024-09-12T13:43:02.472+00:...	file1.json
3	> [{"Device Id": "iot_device_003", "Device Type": "A03", "timestamp": "2024-09-03T12:10:00Z", "location": {"latitude": 40.7128, "longitude": -74.006, "temperature": 22.8, "humidity": 55, "pressure": 1012.8}	2024-09-12T13:43:02.472+00:...	file1.json
4	> [{"Device Id": "iot_device_004", "Device Type": "A04", "timestamp": "2024-09-03T12:15:00Z", "location": {"latitude": 42.3601, "longitude": -71.0589, "temperature": 20.5, "humidity": 70.2, "pressure": 1014.1}	2024-09-12T13:43:02.472+00:...	file1.json

Figure 4. Bronze layer table, containing each event in a separate row as a string.

3.2 Silver layer

Silver layer contains conformed, cleaned, transformed data. This often means removing duplicates, handling corrupted and wrong data. Data might be split into separate columns, and data types are assigned on this layer. Validations and constrains might be added on this layer, quarantine tables could also be added, for data that does not meet given criteria. Figure 5 shows the same data as the previous figures, but transformed. Json structure that previously has been in a single column as a string type, has been parsed into separate columns and the correct data types have been assigned. The meta-data about the original source data has been removed.

	A ^B device_id	A ^B device_type	🕒 timestamp	1.2 latitude	1.2 longitude	1.2 temperature	1.2 humidity	1.2 pressure
1	iot_device_001	A01	2024-09-03T12:00:00.000+00:...	37.7749	-122.4194	25.6	60.2	1013.25
2	iot_device_002	A02	2024-09-03T12:05:00.000+00:...	34.0522	-118.2437	24.1	65.1	1015.3
3	iot_device_003	A03	2024-09-03T12:10:00.000+00:...	40.7128	-74.006	22.8	55	1012.8
4	iot_device_004	A04	2024-09-03T12:15:00.000+00:...	42.3601	-71.0589	20.5	70.2	1014.1

Figure 5. Silver layer table, with data split into the right columns and with correct data types assigned.

3.3 Gold layer

Gold layer has data for a specific use-case, data needed for that use-case can be selected from multiple different tables and even external sources. Different kinds of aggregations and business transformations can be performed on this layer. Those use-case specific tables are primarily used for reporting and ad-hoc analysis. Since many different business users might have access to this layer, any sensitive information should be omitted at this stage. Data modeling like star schema is implemented on this layer.

Figure 6 shows two tables, the table on the left shows a gold layer table, with device_id, device_type_name and temperature. The device_type_name did not exist in the original data; it has been added on this layer. This column has been added, based on the dimension table, visible on the right side of the figure. The business user would not get much information from device_type field, therefore join operation has been conducted, and the original field name device_type has been replaced with device_type_name.

	A_C^B device_id	A_C^B device_type_name	1.2 temperature
1	iot_device_001	Sensor A	25.6
2	iot_device_002	Sensor B	24.1
3	iot_device_003	Sensor C	22.8
4	iot_device_004	Sensor D	20.5

	A_C^B device_type	A_C^B device_type_name
1	A01	Sensor A
2	A02	Sensor B
3	A03	Sensor C
4	A04	Sensor D

Figure 6. Two gold layer tables, a device table and sensor dimension table.

Figure 7 shows different use-case specific gold layer table, a table which only contains devices located in North America.

	A_C^B device_id	1.2 pressure	1.2 latitude	1.2 longitude
1	iot_device_003	1012.8	40.7128	-74.006
2	iot_device_004	1014.1	42.3601	-71.0589

Figure 7. Gold layer table, containing only devices from North America.

The figure 8 graph shows the process that has been described in this chapter. The first table located at a very left side of the figure, contains raw data, each physical json file is one row in that table. Data is read from that table, and each event from within original json file is split into a separate row. Silver layer table that follows table `bronze_publish`, has the data types set and properties of each event are split into separate columns.

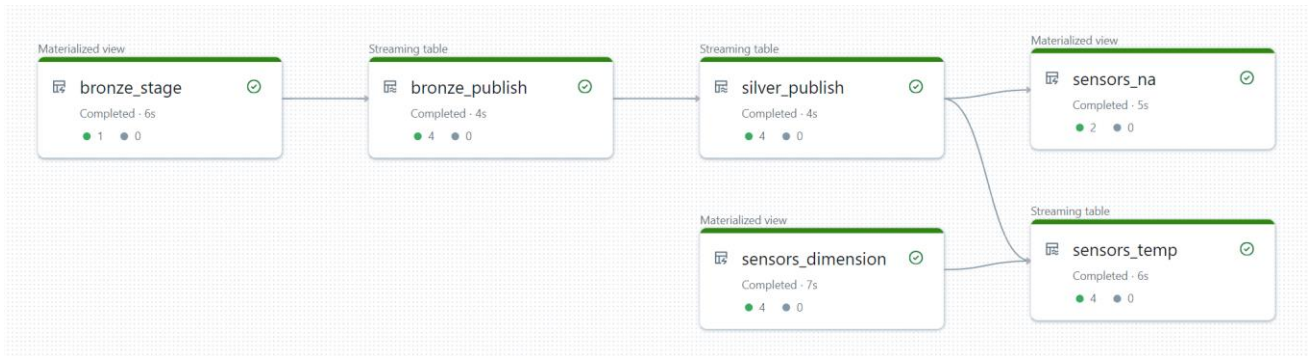


Figure 8. Data lineage graph.

The gold level tables: sensors temperature and sensors North America are created based on silver layer sensors table, Additionally sensors' temperature table is a simple example of star schema modelling, the table is created by joining fact table (`silver_publish`) and dimensions table.

4 Solution overview

4.1 Introduction

Medallion Architecture implementations are based on cloud and cloud computing platforms. There are many reasons why, the primary ones are lack of up-front costs, scalability and flexibility. Lack of upfront costs means, that an organization does not need to purchase an entire data centre to get started with their operations. At any given point a request for more services and resources can be made, which is inevitable as the organization grows. It is also effortless to scale up or down ongoing operations. Services can scale up and down, in an automated manner ensuring the organization's services stay available and cost efficient.

Organizations tend to collect more and more data, while keeping the historical one. Therefore, there is constant need for scaling, and most companies choose cloud providers to meet their data storage needs. One of the services offered by cloud providers are Data Lakes, which are an excellent base for big data workloads.

4.2 High-level overview

The entire data platform is hosted in the Azure cloud, the data storage, compute and orchestration. On a really high-level the data flow looks as follows: data is gathered from source systems, saved in Data Lake. That Data Lake is designated as a staging location, from there the data is read to be further processed. The processed data is saved in separate Data Lake, it's stored using specific format and solution: Delta Lake. (Figure 9).

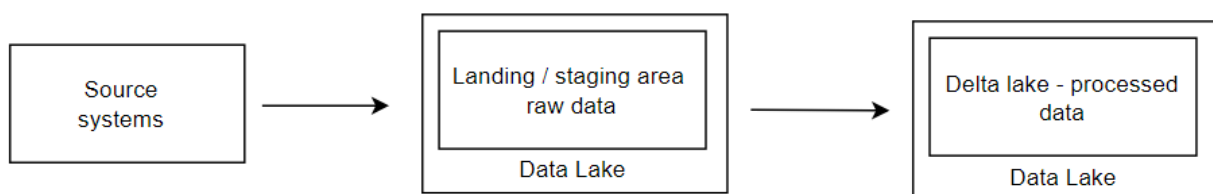


Figure 9. High-level data workflow of the data platform.

4.2.1 Data ingestion

The first part of data platform responsible for the data ingestion looks as follows: there are services and scripts fetching the data from the source systems. They save the data in raw formats in the Data

Lake which is used as landing zone. Some data might be sent from source systems via different middle-ware layers or directly into landing Data Lakes (Figure 10).

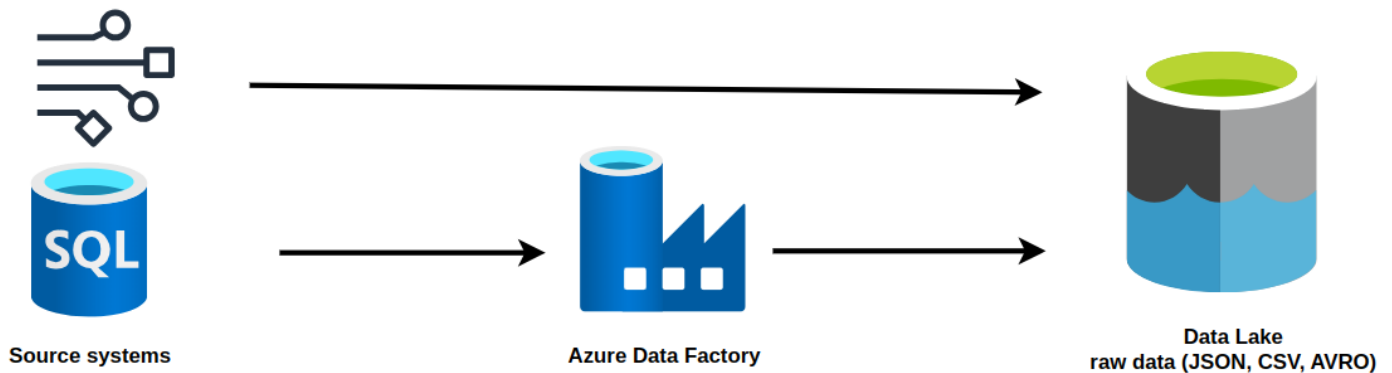


Figure 10. Example data ingestion workload. Data streamed into Data Lake and data being fetched from SQL server using Azure Data Factory and saved in Data Lake.

The left side of the Figure 10 represents source systems which send the data or which the data is being extracted from. The mobile app events are saved directly into the Data Lake, and for data located in SQL server a tool called Azure Data Factory (ADF) is used. Within ADF a SQL query is specified which gets executed and returns results. The results are saved in csv files in the Data Lake.

Data Lakes themselves, are excellent bases for big data workloads and modern data platforms. Data lakes can be described as a centralized data repository, that can hold structured, unstructured, semi-structured data at any scale. Data Lakes offered by cloud providers support tens of thousands concurrent transactions, and basically unlimited storage capacities. (Amazon AWS s.a.)

4.2.2 Data processing

The next layer of data platform is the data processing and transformation layer. In this layer the actual Medallion Architecture is implemented. Medallion Architecture can be implemented in different ways, in the case of the data platform, raw data files are read and saved in Delta Lake solution. Inside cloud computing platform Databricks tables are created which point to the physical data files (Delta Lake files). The tables created in Databricks do not store the data themselves, whenever users want to read data from created tables, the data is actually read from the Delta Lake. What Databricks does it provides the interface for users to query the data, and provisions the virtual machines which actually read and compute the underlying data.

The silver layer is created by reading the bronze layer tables, and processing read data. Processed data is saved in Delta Lake and a table within Databricks is created. Gold layer tables are created in similar way, by reading the needed data from silver layer tables (Figure 11).

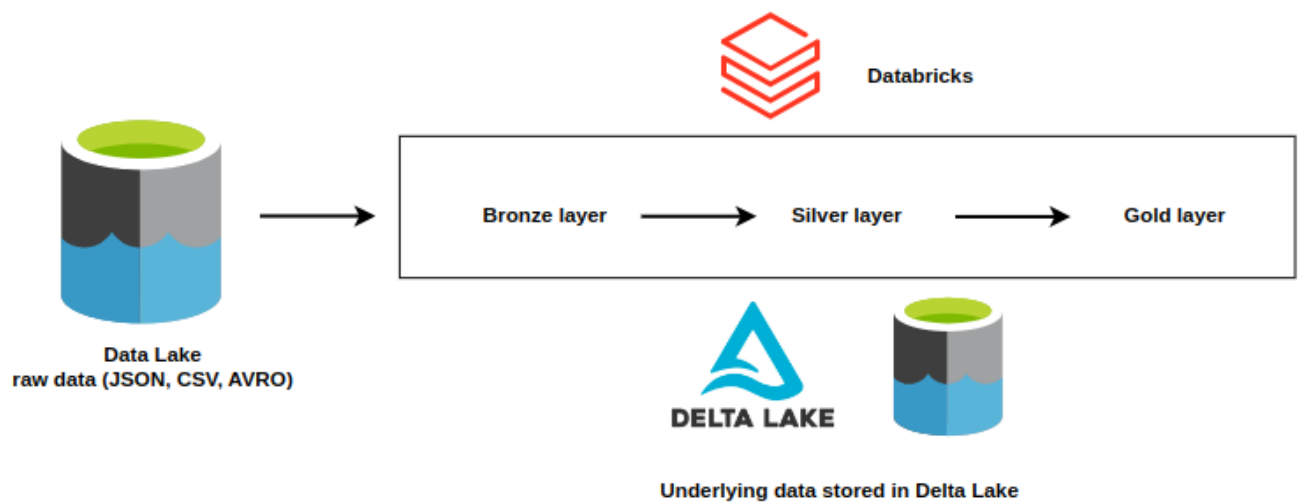


Figure 11. Medallion architecture implemented based on Delta Lake and Databricks.

4.2.3 Serving and governance

Serving and governance layer defines who and how can access a given data. In general, only internal data team has access to all layers of Medallion Architecture, as it might include sensitive information. The business users and other data analysts have only access to prepared and cleaned data from a gold layer. Furthermore, a principle of least privileges is used, business user have only access to the data that is relevant to their department. The solution used for permissions control and governance is called Unity Catalog. It's an open source solution developed and maintained by Databricks. It has been designed to unify a governance across multiple environments in one place. It provides access controls, both on a table and column level, as an example: shipping and finance departments could have access to table with orders. However, only finance users have access to value of order – if data is queried by shipping department the column would show “REDACTED” as a value. Unity Catalog also has features which help tract changes and show lineage of data.

The data can be queried via Databricks, reporting tools and operational systems. The most common reporting use-case is creation of dashboards in the PowerBI. The data for those use-cases is taken from gold layer of medallion architecture. The overview of the entire solution is shown in Figure 12.

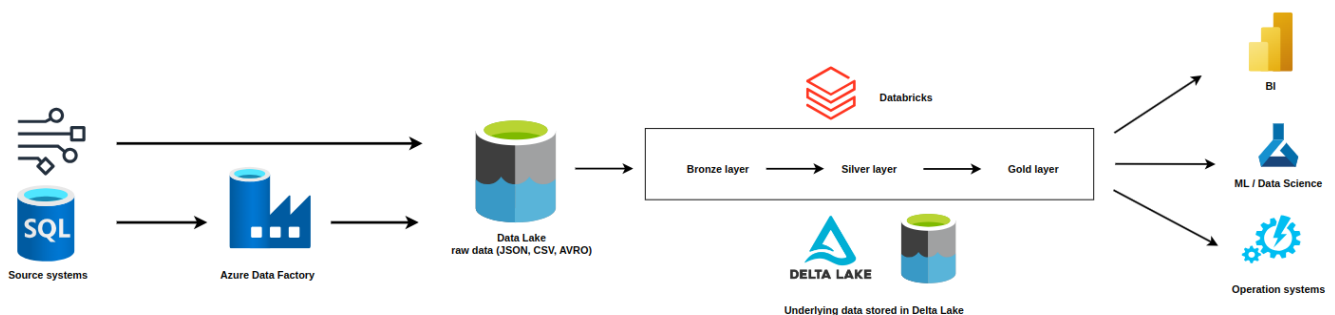


Figure 12. Simplified high-level overview of the solution architecture.

5 Solution implementation

One of the objectives of this thesis is to showcase the practical implementation of the medallion architecture, based on the data platform developments. It is achieved by walking the reader through the entire process, from getting raw data from source systems to business use-cases. There are two specific use-cases, and data sources that have been integrated into data platform, as a part of this thesis. The two data sources in question are mobile application events and on-premises SQL server.

5.1 Data ingestion layer

As previously mentioned in chapter 4.2, data ingestion layer is the layer responsible for fetching or receiving data from various source systems. Gathered data is being saved in a landing Data Lake, from there data is read and processes in data processing layer.

5.1.1 Mobile applications events

Due to the number of users, there are thousands of mobile applications events flowing into the data platform every second. Those events include technical debugging information such as error messages, and events generated based on user activity. There are many different ways to approach real-time data, the approach chosen for the data platform was use of middleware to combine individual mobile application events into micro-batches and send them to the landing Data Lake.

The middleware has been develop using Kafka, an open-source distributed system, designed for handling streaming data. The middleware combines events into json files with multiple events per file. In this particular case micro-batching increases overall throughput and efficiency. (Kafka Apache s.a.)

The entire middleware layer and the data platform are hosted in different Azure subscriptions. It means that there is no direct connectivity between those subscriptions. Middleware needs to send data to landing Data Lake build on top of a Azure Storage Account in a secure way. The default configuration of Storage Account allows access to the data from anywhere via internet. This is not recommended approach; Microsoft recommends restricting access to only selected networks and services. (Microsoft 2023a) To comply with best security practises, public access to the storage is disabled and only particular virtual network from middleware subscription has been whitelisted. In order to make a connection between that virtual network and a storage account a private endpoint has been used (Figure 13). Which is a network interface, that is used to connect private ip address from virtual network to a given service. (Microsoft 2023b)

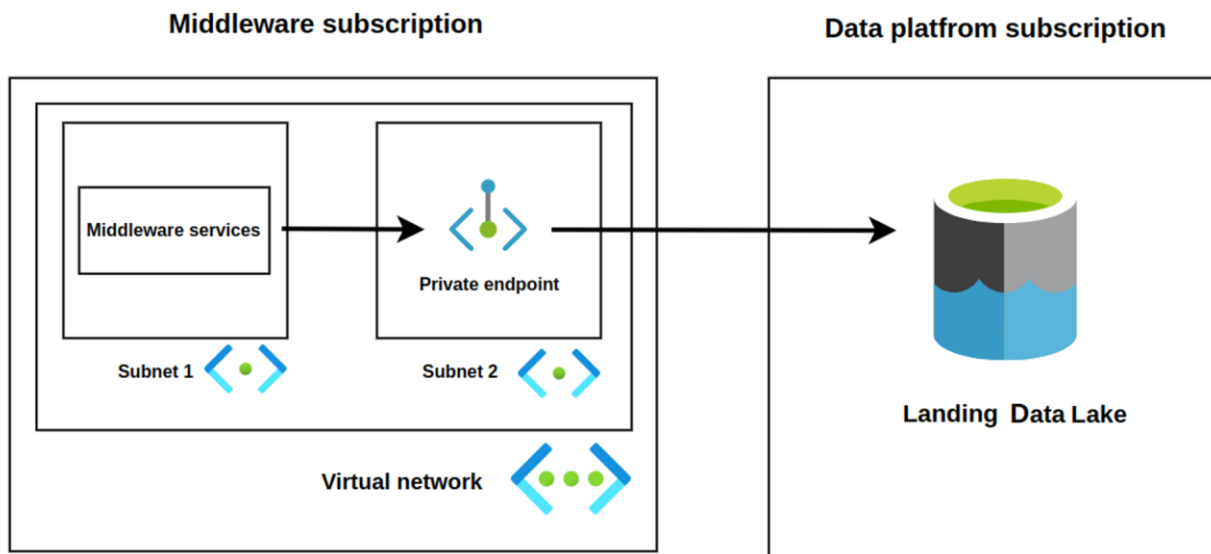


Figure 13. Private endpoint connection to Data Lake, based on “Use private endpoints for Azure Storage”. (Microsoft 2023b)

Many different mobile application events are sent to the landing Data Lake, each event has specific container made for it (Figure 14). Events from each day are stored in separate subfolders. Such structure ensures ease of troubleshooting, and allows straightforward implementation of custom data retention policies. In the case of the landing Data Lake, build-in Azure Storage Account retention policies are used.

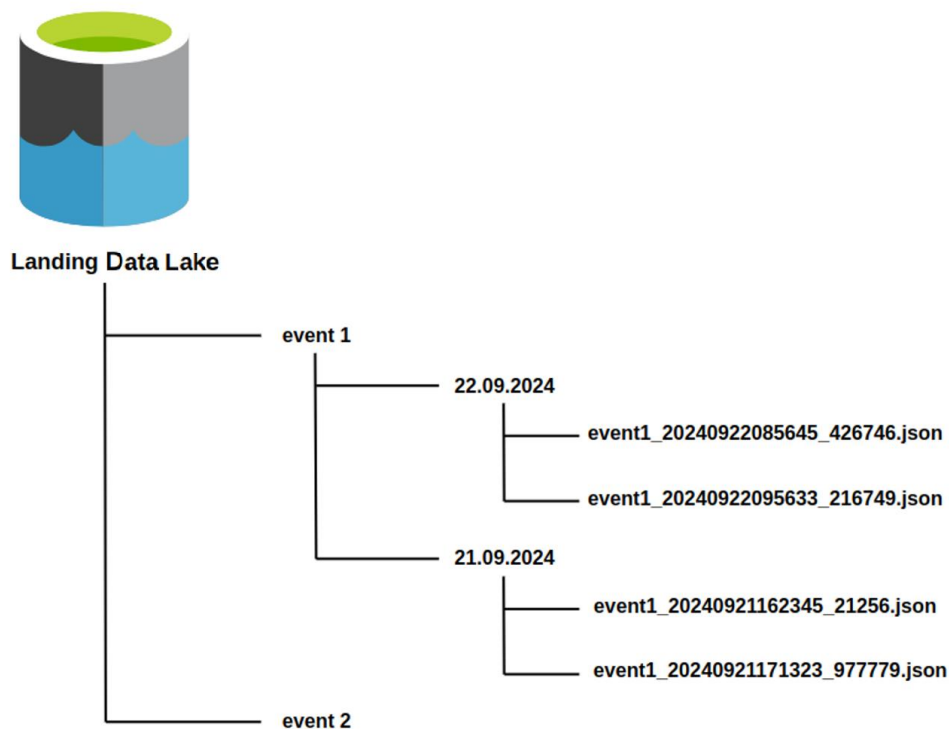


Figure 14. Directory structure of landing Data Lake for mobile application events.

Data retention policies can be described as practises of storing and disposal of data. It's conducted for business and legal reasons. (Porter 2024) Business reason can be a cost saving effort, after the raw data has been converted into more efficient file format, there might not be a reason to store the original files. Compliance with the law, depending on the data, might require implementation of data retention policies.

5.1.2 SQL server

Another data source, that has been integrated into the data platform, is a SQL server with multiple databases. This particular SQL server, contains data related to manufacturing process, and is located outside Azure eco-system. The connectivity to this SQL server is highly restricted and connection is possible only from selected networks. New data needs to be fetched from one of the databases, on a daily basis.

The service used for extracting the data from the database is Azure Data Factory (ADF). The underlying integration runtime that executes the queries onto database is hosted on a virtual machine. The virtual machine is attached to private virtual network, that has in-direct connection to SQL server (Figure 15).

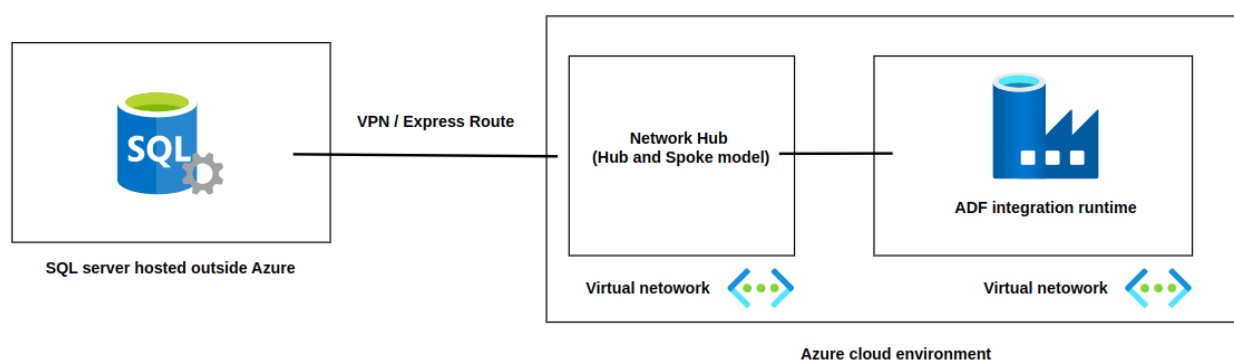


Figure 15. Connection from ADF to SQL server hosted outside Azure.

Tables fetched using ADF from the SQL databases, contain a column: “date_modified” which contains last date when given record has been modified or inserted. Pipeline that fetches the data runs specific query, which returns only records, where column “date_modified” meets specific criteria. Data is then saved in raw format within a landing Data Lake. For this specific use-case and those specific tables, data is fetched once a day and saved in csv format. The pipeline is scheduled to run every day, couple of minutes after the midnight and retrieve data from the previous day.

5.2 Data processing layer

After the data has been saved in the landing location, it can be read and processed. A broad definition of data processing can be stated as follows: data processing is a process of transforming raw data into structured form, suitable for analysis, data science and decision-making. Transformations can be a complex process with multiple steps, some of those steps might include: decoding, filtering, cleaning, type conversion, aggregation, exceptions handling and feature engineering. The end result is supposed to be data that, end-users can trust and bring value out of.

During the process of data transformation data is read, concatenated, manipulated many times over and over again. Source file formats such as json and csv are suboptimal for big data workloads. Nowadays many big data workloads are based on solutions build on top of Apache Parquet file format. In case of the data platform, that solution is Delta Lake.

5.2.1 Apache Parquet and Delta Lake

Apache Parquet

Apache Parquet is open-source columnar data format, that has been designed for efficient large-scale data storage and retrieval. Parquet is a columnar data format, which means it's column oriented, values for each column are stored together. (Apache Parquet s.a.) The basic concept is to

allow faster data reads: the queries only read relevant columns and do not read all of the rows. Such a design also allows better compression, as in most cases data within a single column, is of similar structure and same data type (Figure 16).

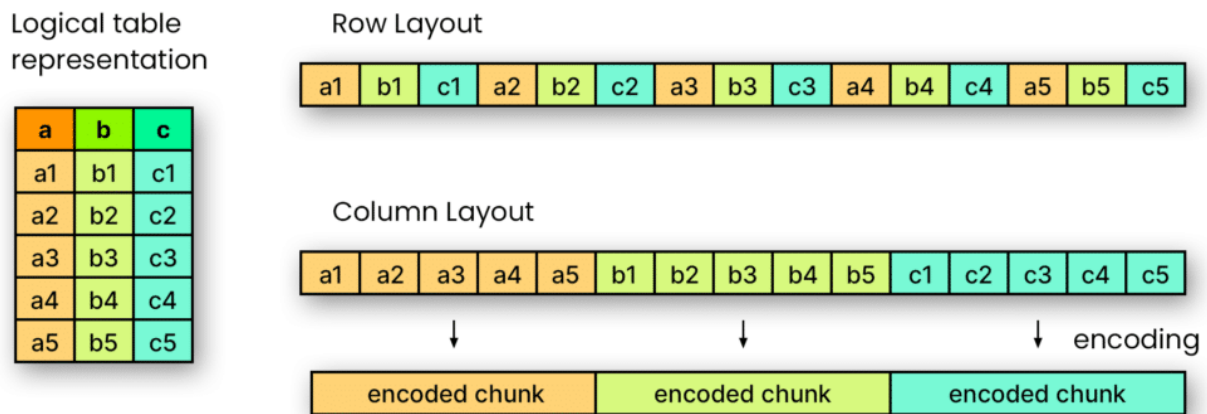


Figure 16. Apache Parquet columnar storage (Dremio s.a.)

Parquet also has row groups, which are blocks of rows. Parquet stores metadata, some of it is at column and row group level. This metadata includes statistics on a column level, like minimal and maximum values, count of null and non-null values.

Parquet uses techniques like predicate pushdown and data skipping, which utilize metadata statistics, allowing faster data reads. To help understand how does those techniques work, one can image a table containing information about cars. Analysts would want to see old cars, cars manufactured before 1981. Like previously mentioned Parquet metadata among other statistics, contains maximum and minimal values of columns on a row group level. What it means is, that only the relevant data blocks will be read, resulting in faster data reads. Blocks of data where minimum value of "manufacturing_date" is more than 1981 will be skipped (Figure17). (Data Mozart 2024)

car_name	manufacturing_date		
car1	2024	row group 1	manufacturing_date min(1960) max(2024)
car2	1960		
car3	2014	row group 2	manufacturing_date min(2014) max(2020)
car4	2020		
car5	1956	row group 3	manufacturing_date min(1956) max(1990)
car6	1990		

Figure 17. Row groups data skipping example.

Following this example (Figure 17), row group 1 would be read, as minimal value is less than 1981. Row group 2 would be skipped as the minimal value is more than 1981 and row group 3 would be read as minimal value is less than 1981.

Parquet among other optimizations and encodings uses dictionary encoding. In the large datasets some values will be repeated many times, in order to save disk and memory space, each unique value is assigned an index. As an example, word “Helsinki” could be assigned a integer index 0, and then all of the original “Helsinki” words would be replaced with its index. This technique is especially effective in dataset with few unique values. (Apache Parquet 2024, Kinetica s.a.)

Delta Lake

Delta lake is an open-source storage layer, build on top of Apache Parquet. It brings ACID transactions, versioning, “time-travel”, efficiency improvements and quality of life features. All of those features are possible due to implementation of transaction log. Transactional log contains metadata about conducted operations, schema, partitions and statistics. (Delta Lake s.a.)

What makes transactional log a translation log, are records of changes made to the table, the operations conducted on the data (Figure 18). Transaction log records those operation in json files. Each json file represents different version of a table and contains only the changes made in that commit. In order to get most recent state of the table previous json files need to be read. In order to avoid many small files, every ten json files, there is a checkpoint made (Figure 19). This checkpoint is a Parquet file, therefore to get recent state, only most recent Parquet file needs to be read and json files that follow it. Additionally, after the transactional log files are read, they are cached to further increase efficiency. (Lee 2024)

version. There are limitations to it, in order to save disk space, older transaction log files get removed. (Delta Lake 2023)

Delta lake has more efficient data skipping compared to Parquet. It's due to the fact that Parquet has metadata inside of the file, which allows to skip particular data blocks. Delta lake has metadata in the transactional log, which allows to skip entire data files. (Delta Lake 2023)

Delta lake has many other important features like Z-ordering, which organizes files based on values in certain columns. Delta Lake allows further optimization, by mitigating small files problem, a problem where many small files, make reads much longer. Delta Lake has optimization command, which compact small files into larger ones. (Delta Lake 2023)

5.2.2 Apache Spark and Databricks

Apache Spark

Apache Spark is open-source engine for large-scale data processing, it allows efficient data processing, as it's designed to process data in parallel. Spark has APIs to different programming languages, which allow end-users, to conduct data operations.

In order to take advantage of parallel data processing, Spark runs in cluster configuration. Data is divided into smaller chunks and assigned to nodes of a cluster. Cluster itself is a collection of computers, in case of Spark there is driver node and worker nodes. The driver node is the one that end-user interacts with, that node divides workload between worker nodes, coordinates execution and result collection (Figure 20).

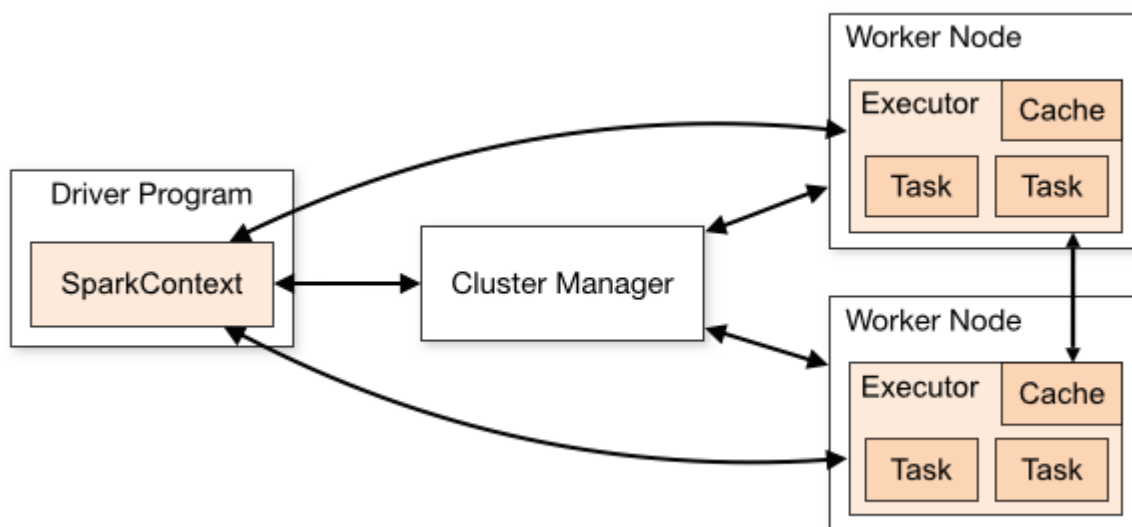


Figure 20. Cluster Mode Overview (Apache Spark s.a.)

Databricks

Databricks is a cloud computing and analytics platform. It means that it orchestrates compute, manages the underlying data and allows analysis on data. Databricks is hosted on top of cloud providers like AWS, Azure or GCP. The code that conducts the transformations can be developed and scheduled inside Databricks, as a part of a data pipeline. What Databricks does is, upon the execution of the data pipeline it provisions virtual machines, in the environment set inside cloud provider. Those provisioned virtual machines are the ones running Spark.

The amount of virtual machines, depends on the configuration of the cluster in Databricks, different sized clusters are needed for different workflows. There are many nuances that go into choosing a right cluster. An example can be a memory size, to ensure optimal execution, processed data should fit in the memory. If the processed data is too big to fit in the memory, a spill occurs, and excess data is written to disk. Spills causes a decrease in performance, as reads and writes to disk are generally slow. Therefore, if possible spills should be prevented, by allocating sufficient amount of memory.

5.2.3 Data processing on the Data platform

The code that conducts the transformations is written in Python and SQL inside interactive files called notebooks, and also sql files. Those files are chained together in a data pipeline, in order to ensure data consistency, execution of each file is depended on the previous file.

Those pipelines are executed on a scheduled basis from within the Databricks or triggered by ADF. Pipelines are executed in a selected Spark cluster, which has been provisioned by Databricks. Code written in Python or SQL uses Spark API to send commands to Spark engine, which conduction data operations. The processed data is then written to a dedicated Data Lake in Delta Lake (Figure 21).

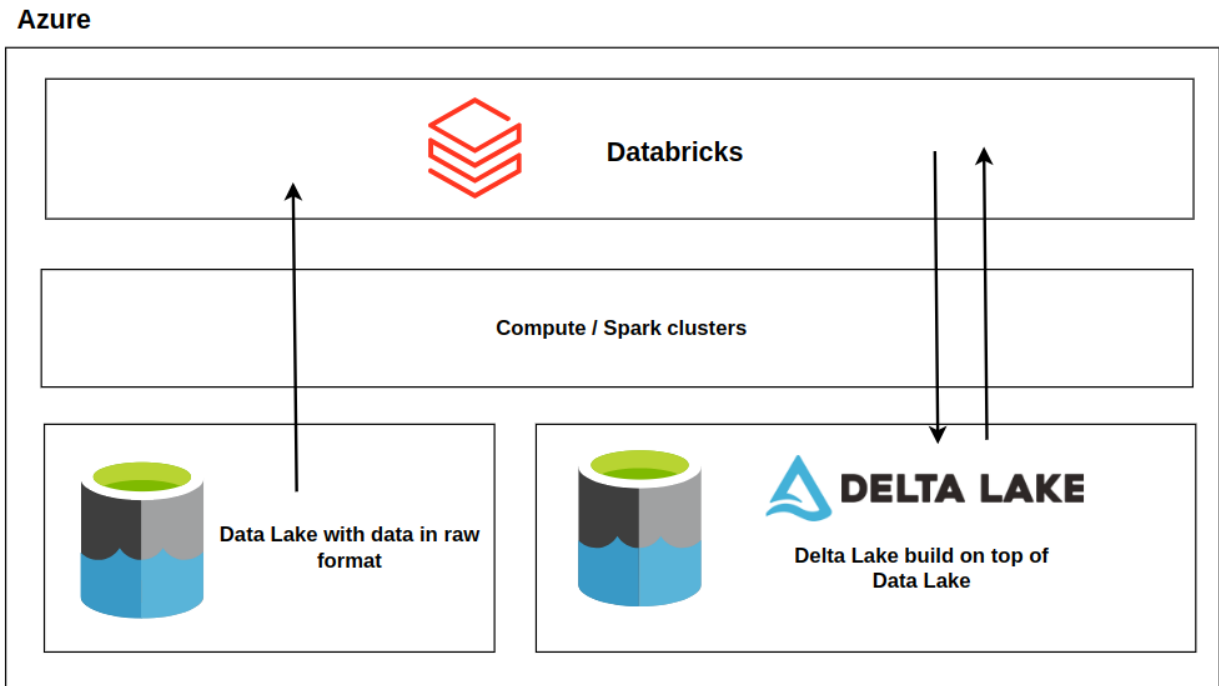


Figure 21. Overview of Databricks, Spark, Delta Lake implementation.

5.2.4 Mobile application events processing

Mobile application events are streamed into landing Data Lake in json files, each file contains couple hundreds events of the same type. Each event type is stored in separate container (Chapter 5.1.1). Data from those files needs to be parsed into separated events and then organized into columns. An example file containing events can have a following structure: the json file contains an array of objects. Each object is a separate event. Mobile events often have a complex structure and contain nested objects within (Figure 22).

```
[
  {
    "event_id": "23",
    "event_type": "debug_log",
    "timestamp": "2024-01-08T12:00:00Z",
    "details": {
      "log_level": "INFO",
      "message": "App launched successfully",
      "network_status": "WiFi connected",
      "device_info": {
        "os_version": "Android 11",
        "app_version": "1.2.2"
      }
    }
  },
  {
    "event_id": "102",
    "event_type": "error_log"
  }
]
```

Figure 22. Example structure of a json file, containing mobile app events.

Bronze layer

The first step is to read the raw data and store it in the Delta Lake as a part of a bronze layer. In this case and for this particular data, there is one bronze layer table for each event type. The table entities exist within Databricks, and are pointing to the underlying Delta Lake. Those tables have json files parsed, so that each event is in the separate row. Those events are stored as a json strings, the columns that follow the event are metadata columns. Those metadata columns can be used for troubleshooting, data validation purposes, data retention policies implementations. Depending on the technologies and logic used, they can be also used for ingestion of newly arrived data into downstream tables. An example of this can be following workload: new data files are loaded from landing Data Lake into bronze layer table. Afterwards the data is read from the bronze layer, with condition specifying, that only data loaded into bronze table, within past 24 hours is read. Only that data is then appended into silver layer table.

Silver layer

There are two silver layer tables for each of the events, the first table can be called silver stage, and it contains events parsed into separate columns. Also in this table, data types are set for columns, with some columns being kept as structured nested objects. Data types are set, either by manually defining schema and then applying it onto json string, or by auto-Infering schema.

The table that follows silver stage, can be called a silver publish, since it will be the table from which gold tables will read data. At this point a data conversions are made, an example of this is, conversion of unix timestamp, into a timestamp of a following format format “YYYY-MM-DD hh:mm:ss” (Figure 23). The column itself gets replaced with a new one, as a data type has changed from integer to timestamp type.

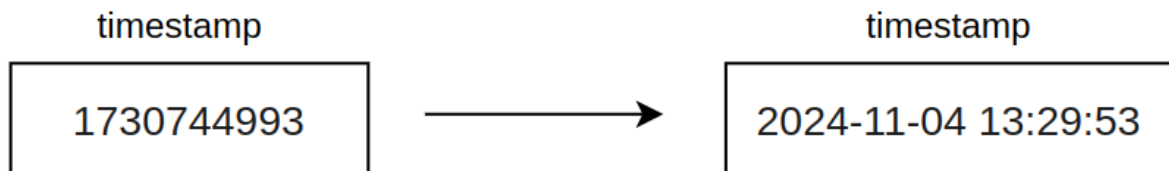


Figure 23. Showcase of timestamp conversion.

Columns get renamed to fit a snake case convention, which uses underscores instead of spaces (Figure 24).



Figure 24. Showcase of column name transformation into camel case.

At this table data cleaning and de-duplication is conducted, there can be different checks set in place to ensure the accuracy and quality of data. In the case of this workload, to deduplicate the data, distinct commands and windowing functions are used.

To validate and ensure the quality of the data a constrains checks can be made, for example if value of a certain column significantly exceeds a average, that row can be put in a special quarantine table. Which then can be reviewed, such process ensures that only accurate data is added to the final tables.

There is no need to conduct any advanced data transformations or calculations on the mobile applications events, the only transformations done are simple string operations. After which the data on the silver publish level is ready to be used for business specific use-cases in the gold layer tables.

Gold layer

In the case of the data platform there are many different gold layer tables, which are made for specific use cases. Those tables are the ones that business users use, to support decision making. Therefore, it is critical to ensure accuracy of that data. An example of a gold layer table can be a table with a number of active users, on a device level. Business users would like to see how many active users there are per specific device type. In order to provide business users with such table, two tables need to be joined: a silver layer table with a specific mobile event and a dimension table. The dimension table is the one with business information for each device type. Information like internal name, version number, cost of a device etc.

Other example business use-case is information about the number of activated devices. The reasoning behind this use case, is that business not only wants to know how many devices were sold. The sales information is easily accessible from an ERP system. What business wants to know, is how many people actually started using devices. One of the events that is send from the mobile application is event, that is triggered whenever a device is connected to mobile application for the first time. The event goes through all of the layers, and at the final gold layer, business can see time-based activations of different devices on a country level. Furthermore, data prepared this way, can with ease be used by analysts and business users to create dashboards, to graphically showcase this data. Most of the business users use PowerBI as their tool for graphically visualize the data. Often dynamic dashboards are created, in which users can adjust parameters like filters on already made reports. Business users with a access to the data, use PowerBI to directly connect to data platform, as it has built-in connector to Databricks. It is also possible to create simple dashboards within Databricks (Figure 25).

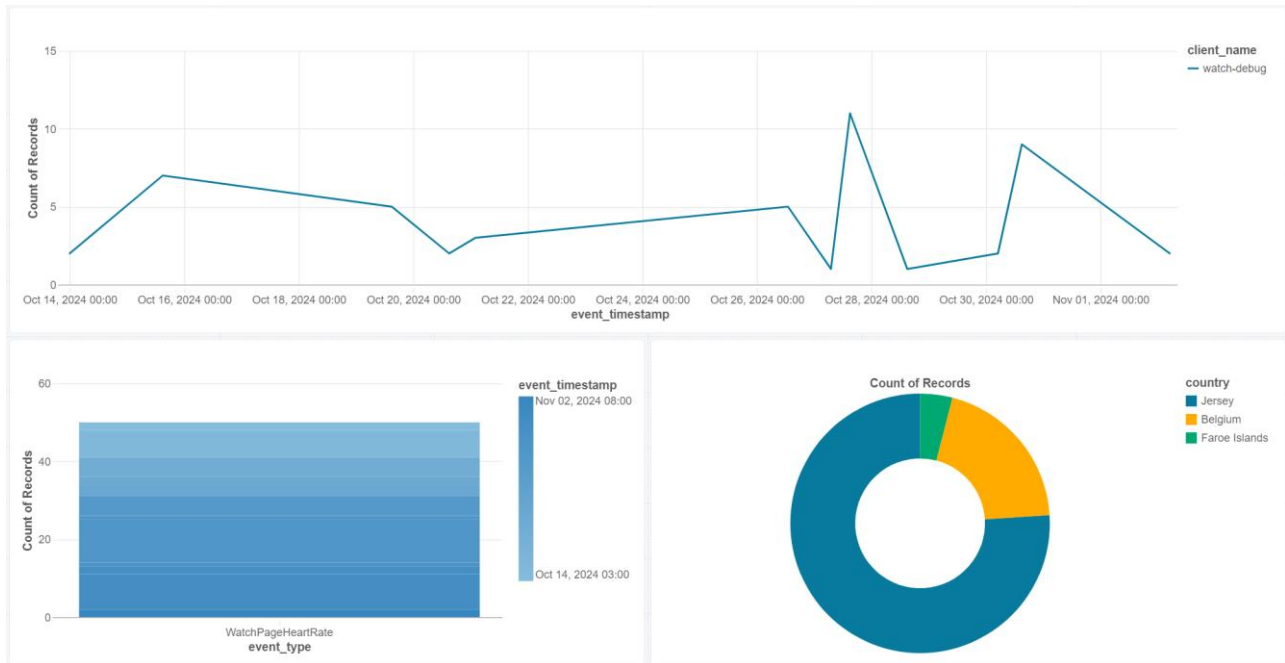


Figure 25. An example dashboard created in Databricks.

Another use-case for gold level activations data is inventory status table. Activations data and data from ERP system, can be used to create such table. It shows, how many devices are still on the shelves of the shops. Device might have been bought by a reseller, but not yet bought and activated by final users. All what would need to be done is combine already existing gold level activations data with ERP system data.

5.2.5 SQL Database processing

Business wants to track information about warranty repairs, a data regarding which devices have been returned for a repair, what was broken. This data is located in in the database on the SQL server outside of Azure eco-system. The process of extracting data from that particular SQL server has been described in-depth in chapter 5.1.2.

One of the tables that is integrated with the data platform is a table with the repair information. This table contains a metadata filled with a information when a given row has been modified. Daily job is scheduled which gathers data based on this information. Repair information contains dates when: a device has been received for a repair, repaired and send back to the customer. Therefore each row is modified multiple times, and because data is fetched daily, based on "last_time_modified", there are duplicates (Figure 26).

repair number	device type	date received	date repaired	date sent	date modified
761	Device 1	21.06.2024	None	None	21.06.2024
762	Device 2	21.06.2024	None	None	21.06.2024
761	Device 1	21.06.2024	25.06.2024	None	25.06.2024
761	Device 1	21.06.2024	25.06.2024	26.062024	26.06.2024
762	Device 2	21.06.2024	27.06.2024	None	27.06.2024
762	Device 2	21.06.2024	27.06.2024	28.06.2024	28.06.2024

Figure 26. A sample data of a repair table, showcasing the changes made.

Bronze layer

ADF is a tool used for fetching the data and saving it in the landing Data Lake (5.1.2). ADF also triggers a Databricks pipeline, the first part of the pipeline reads the new file based on timestamp included in the file name. The read data is appended to the bronze table, along with metadata: time of data being appended, and original file name. There is no filtering logic implemented on the bronze table, and because each row has been fetched multiple times there are duplicates on this layer. On this layer this is not an issue, and it might be useful for very specific use cases.

Silver layer

On the silver layer the data types are set, and the duplicates are removed. In this particular case a windowing functions is used. Data is aggregated on a repair number level, and out of the records with the same repair number, the ones with latest timestamp of insertion into bronze layer are kept. In the case of an example data shown in figure 26, the result of such function would be as in figure 27.

repair number	device type	date received	date repaired	date sent	date modified
761	Device 1	21.06.2024	25.06.2024	26.062024	26.06.2024
762	Device 2	21.06.2024	27.06.2024	28.06.2024	28.06.2024

Figure 27. Repair data from Figure 27, after the windowing function.

Gold layer

The gold layer tables based on repair data, are use-case specific. Some of those use-cases include, dashboards with information about how many devices have been send to repair. It provides a

valuable insight to the business, as it allows to see which parts and elements of devices are failing. Which in direct way can contribute to improvements of produced products, and therefore also to sales increases.

Hypothetically combining the mobile application events and the repair data could lead to even deeper insights. As an example, an error codes that the device is sending via mobile application could be used to get better understanding why certain devices have failed.

5.3 Serving and data governance layer

Data governance is its own discipline, focused on ensuring security, integrity and availability of the data. (IBM 2024) Governance layer is needed to define rules how the data should be handled, how and by who it can be accessed and how it should be stored. Without proper data governance layer managing the data and access to it, becomes difficult.

5.3.1 Access control

In case of the data platform data, data is served to end-user via Databricks. Within Databricks a open-source solution Unity Catalog is used for data governance (chapter 4.2.3). In general: only internal data team has access to all layers of medallion architecture. Those are the people which need to have access to the raw data. Every other employee has only access to the data that they explicitly need, and they have permission to use. Business users have only access to gold layer data. As mentioned in chapter 3.3, the gold layer tables are use-case specific, and they are highly transformed. The personal information, being the information that can lead to identification is highly sensitive, therefore it's left out of this layer or anatomized. For example, the exact dates are round up and the aggregates are made. Making it impossible to pinpoint any particular device.

Those rules are enforced by assigning users to right groups, each group has access to specified tables. There are groups made for departments and specific tables, for example there is group made for finance department. Users assigned to this group have access to multiple tables, however there might another group called partner reporting access, that group would have access only to subset of tables that finance department has access to. Permissions to tables are not assigned individually, but always to a group. Its considered a good practise,as it simplifies management and ensures consistency.

5.3.2 Data consumption

Most of the business users lack the SQL knowledge and proficiency, and most use analytical tools such as PowerBI to access the data (Chapter 4.2.3). The connection from those tools is made directly to Databricks, which provisions virtual machines, which read the underlying data and return it back to the user. Using that data comprehensive dashboard and visualizations are made, to provide vital business insights, taking the organization to new heights. Those insights would not be possible without having a centralized data platform.

Each compute unit that is used whenever users make queries to read underlying data, is assigned to a specific group. It allows for easier usage and costs tracking, each compute unit has a special tag assigned which can be tracked from a cost analysis dashboard in Azure.

6 Discussion

6.1 Objectives

The goal of this thesis has been to bring value to the business, by developing an in-house data platform. This way business can conduct more extensive and comprehensive analysis, which leads to better, data-driven decisions. Organization also wanted to reduce overall operational costs, by switching from many third-party platforms, into an in-house one.

Those goals have been successfully achieved, as data analysts and business users, have access to data that previously has been inaccessible, or access was limited due to the usage of third-party solutions. Many different analyses have been made possible, as data platform has become a centralized repository with data from different operating systems. Therefore, more in-depth, more comprehensive and efficient analysis, have been made possible.

The organization still uses some of the third-party solutions, however operational costs have been significantly reduced. In total, expenses have been decreased by over 50%. However, operating the data platform still does cost, the biggest expense being a qualified personnel needed for development and maintenance of the data platform. After which the biggest expenses are compute and storage.

The computing costs consist of cost of running the virtual machines in the Azure, and the markup of the cloud computing platform Databricks. There is not that much that can be done about those costs, other than trying to optimize the data pipelines and getting into custom agreements with cloud providers. Storage costs will grow, as new data is gathered and historical one is kept. In the case of the Medallion Architecture this can become a significant concern, as the same data is stored multiple times.

6.2 Medallion Architecture evaluation

Medallion Architecture itself, is a proven concept that brings great possibilities for data teams. The ability to have data at different stages of processing is valuable. It brings more structure to entire data platform and development process. Keeping the silver layer tables as clean data, and combining it with different tables in gold layer, provides a structured layout. Keeping the bronze and silver layers locked down, while granting the access to gold layer data, ensures governance structure and consistency. The gold level tables are made for specific business use-cases, only the data that will be needed for that use-case is selected from the silver layer. This approach guarantees the integrity of governance and security principles, as no sensitive information is visible on a gold layer.

As mentioned, storage can become a concern, as the same data is stored multiple times. In order to keep storage volumes under control, Medallion Architecture should be implemented according to needs of a given organization. Perhaps keeping just raw data files as a bronze layer, and only creating one table in the silver layer, is sufficient for some organizations.

Delta Lake and Medallion Architecture are often used together, as in recent times most Medallion Architecture implementations are done using Delta Lake. However, Delta Lake has been adopted as a Big Data storage solution, far beyond just Medallion Architecture.

Delta Lake as storage solution, works incredibly well as a storage solution in the Data Platform. The data retrieval happens in a timely manner, and optimization and version features of Delta Lake are used on daily basis by organization's data team.

6.3 Reflections on implementation

There are some intermediary tables within layer of Medallion Architecture implementation in the data platform, that in retrospective are redundant. In most use-cases there is no need to have two tables on bronze layer. Some implementations of Medallion Architecture do not load the raw data into Delta Lake and keep raw files in their original format as a bronze layer. In case of the data platform, for some use-cases, the original source files are kept, and they are also load into Delta Lake. Making it, so that in practice, the same raw data is stored three times: original data files, bronze stage table and bronze publish table. This is clear inefficiency, not caused by the Medallion Architecture itself, but rather implementation of it, on few particular use-cases.

Other than overuse of intermediate tables, there are no clear misuses or wrong implementations of selected technologies. If the given problems and data sources were presented again, author with full confidence would recommend the same technology stack.

However, there are other comparable technologies that could be used for implementation of data platform. Instead of Azure, data platform and Databricks could be hosted on different cloud platforms like AWS or GCP. It would involve usage of different data ingestion and storage solutions. Another approach would be to instead of Medallion Architecture, use modern data warehouse such as snowflake.

6.4 Further studies

Further project and research work could be conducted, by implementing Medallion Architecture using different technologies. Such academic work would provide deeper insight, in different ways on how to implement Medallion Architecture. Different studies could also be conducted implementing the

same workloads on Delta Lake using Medallion Architecture and selected modern data warehouses. Outcome of such a study could provide an answer, which technology stack is better for a given use-case. Out of scope of this thesis, was also to get more in-depth into inner workings of Delta Lake, however a dedicated research work could be conducted on Delta Lake and possible improvements of it.

6.5 Learnings & self-evaluation

Before starting the thesis process, the author had experience in developing data pipelines, as he already has been developing the data platform. The author already had an understanding of data engineering concepts and proficiency in usage of technologies such as Spark, Delta Lake.

This thesis allowed the author to structure and consolidate his knowledge, as well as to delve deeper into inner workings of technologies used. The author researched how does Apache Parquet and Delta Lake formats work underneath. Furthermore, the author structured and expanded his knowledge about traditional data warehouses.

The implementation of the solution itself allowed the author to increase his technical prowess and competencies. As the mobile application events are high volume data, it required special considerations, such as right compute selection and ensuring the pipelines themselves are optimized and efficiently process the data.

The author discovered advantages and disadvantages of Medallion Architecture, as well as drawbacks of the solution implementation itself, as it already has been said, for high volume events there should not be any bronze layer tables. It should be enough to keep the raw files as bronze layer, there is no need to store both raw files and raw data in Delta Lake. The same data is already stored multiple times, and there is no need to store raw data twice. The approach to store raw data in the original data files, and on the bronze layer using Delta Lake is a good approach, and has its justification. However, for high-value events and large quantity of data, in the author's opinion there is be more disadvantages then advantages.

To conclude, throughout the thesis process, the author has approached new problems, which lead him to refining and improving his skill set. Writing the thesis allowed the author to structure and deepen author's knowledge. It also made the author realize the drawbacks of the solution, and how to mitigate some of them. Overall, the outcome of a thesis has been a huge success. The analytical capabilities of the organization has been greatly increased, as all of the data source has been successfully integrated.

References

Amazon AWS s.a. What is a Data Lake? URL: <https://aws.amazon.com/what-is/data-lake/>. Accessed 14.09.2024.

Apache Kafka s.a. Introduction. URL: <https://kafka.apache.org/intro>. Accessed 22.09.2024

Apache Parquet s.a. Apache Parquet. URL: <https://parquet.apache.org/>. Accessed 04.10.2024.

Apache Parquet 2024. Encodings. URL: <https://parquet.apache.org/docs/file-format/data-pages/encodings/>. Accessed 04.10.2024.

Apache Spark s.a. . Cluster Mode Overview. URL: <https://spark.apache.org/docs/latest/cluster-overview.html> . Accessed 15.10.2024.

Database Guide 2016. What does ACID mean in Database Systems? URL: <https://database.guide/what-is-acid-in-databases/> Accessed: 05.10.2024.

Databricks 2024a. Data Lakehouse. URL: <https://www.databricks.com/glossary/data-lakehouse>. Accessed 08.09.2024.

Databricks 2024b. Medallion Architecture. URL: <https://www.databricks.com/glossary/medallion-architecture> . Accessed 08.09.2024.

Databricks s.a. Star Schema. URL: <https://www.databricks.com/glossary/star-schema>. Accessed 05.11.2024.

Data Mozart 2024. Parquet file format – everything you need to know! URL: <https://data-mozart.com/parquet-file-format-everything-you-need-to-know/>. Accessed: 04.10.2024.

Delta Lake s.a. Build Lakehouses with Delta Lake. URL: <https://delta.io/> Accessed 05.10.2024.

Delta Lake 2023. Delta Lake vs. Parquet Comparison. URL: <https://delta.io/blog/delta-lake-vs-parquet-comparison/>. Accessed 05.10.2024.

Dremio s.a. What is Apache Parquet? URL: <https://www.dremio.com/resources/guides/intro-apache-parquet/>. Accessed 04.10.2024.

IBM 2024. What is data governance? URL: <https://www.ibm.com/topics/data-governance>. Accessed 22.10.2024.

IBM s.a. What is ETL (extract transform, load)? URL: <https://www.ibm.com/topics/etl>. Accessed 26.10.2024.

IDC 2024. Worldwide IDC Global DataSphere Forecast, 2024–2028: AI Everywhere, But Upsurge in Data Will Take Time. Statista. Accessed 16.09.2024.

Karmani, M., Mustafa, G., Sarwar, N., Wajid S., Nasir, J., Siddque, S. 2020. A Review of Star Schema and Snowflakes Schema

Kinetica s.a. Dictionary Encoding. URL: https://docs.kinetica.com/7.1/concepts/dictionary_encoding/. Accessed 04.10.2024.

Lee, D. 2024. A peek into the Delta Lake Transaction Log. URL: <https://dennyglee.com/2024/01/03/a-peek-into-the-delta-lake-transaction-log/> Accessed 05.10.2024.

Microsoft 2023a . Storage Accounts and security. URL: <https://learn.microsoft.com/en-us/azure/well-architected/service-guides/storage-accounts/security>. Accessed 27.09.2024

Microsoft 2023b. Use private endpoints for Azure Storage. URL: <https://learn.microsoft.com/en-us/azure/storage/common/storage-private-endpoints>. Accessed 27.09.2024

Oldham, Ed. 2023. Medallion Architecture: What is it? Advancing Analytics. URL: <https://www.advancinganalytics.co.uk/blog/medallion-architecture>. Accessed 13.09.2024.

Oracle s.a. What is a Data Warehouse? URL: <https://www.oracle.com/database/what-is-a-data-warehouse/>. Accessed: 26.10.2024

Porter, A. 2024. What Is Data Retention? Implementing Effective Practices. URL : <https://bigid.com/blog/what-is-data-retention/>. Accessed 20.09.2024.

Rehman, K., Ahmad, U. & Mahmood S., 2018. A Comparative Analysis of Traditional and Cloud Data Warehouse.

Appendices

Appendix 1. List of figures

Figure 1. Example star schema. (Databricks s.a. a)	7
Figure 2. Medallion architecture. (Databricks 2024 b)	9
Figure 3. Bronze layer table, with contents of entire json file in each row.	10
Figure 4. Bronze layer table, containing each event in a separate row as a string.....	10
Figure 5. Silver layer table, with data split into the right columns and with correct data types assigned.	10
Figure 6. Two gold layer tables, a device table and sensor dimension table.	11
Figure 7. Gold layer table, containing only devices from North America.	11
Figure 8. Data lineage graph.....	12
Figure 9. High-level data workflow of the data platform.	13
Figure 10. Example data ingestion workload. Data streamed into Data Lake and data being fetched from SQL server using Azure Data Factory and saved in Data Lake.	14
Figure 11. Medallion architecture implemented based on Delta Lake and Databricks.	15
Figure 12. Simplified high-level overview of the solution architecture.	16
Figure 13. Private endpoint connection to Data Lake, based on “Use private endpoints for Azure Storage”. (Microsoft 2023b)	18
Figure 14. Directory structure of landing Data Lake for mobile application events.	19
Figure 15. Connection from ADF to SQL server hosted outside Azure.....	20
Figure 16. Apache Parquet columnar storage (Dremio s.a.).....	21
Figure 17. Row groups data skipping example.....	22
Figure 18. Json file with commit information: type of changes made, size of the changes made, statistics, pointers to Parquet files.	23
Figure 19. Delta Lake file structure, data files on left, transactional log on right.	23

Figure 20. Cluster Mode Overview (Apache Spark s.a.).....	24
Figure 21. Overview of Databricks, Spark, Delta Lake implementation.	26
Figure 22. Example structure of a json file, containing mobile app events.....	27
Figure 23. Showcase of timestamp conversion.	28
Figure 24. Showcase of column name transformation into camel case.	28
Figure 25. An example dashboard created in Databricks.	30
Figure 26. A sample data of a repair table, showcasing the changes made.	31
Figure 27. Repair data from Figure 27, after the windowing function.....	31